Busca digital

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Árvore digital
- Árvore digital binária
- Árvore Patrícia

Introdução

- Problema geral da busca supõe conjunto de chaves $S = \{s_1, \ldots, s_n\}$
  - Cada chave é indivisível: tamanho não excede 1 palavra
    - Armazenamento é eficiente
  - Todas chaves possuem mesmo tamanho (bytes)
    - Caso contrário: menores são completadas com bytes extras
  - Comparação de duas chaves: exige uma única comparação de palavras do computador

Introdução

- Exemplo de problema: armazenar e localizar frases de um texto literário
  - Cada frase é tratada como uma chave de busca
    - Tamanho das chaves varia demais e excede demais palavra do computador
    - Cada chave (frase) pode ser decomposta em partes (caracteres)
    - Chaves podem compartilhar prefixos
      - Armazenamento e busca individual é ineficiente
  - Busca digital: mais adequada para essa situação

Árvore digital

- Busca digital: assume que cada chave é composta por conjunto de caracteres de um alfabeto
  - Comparações entre chaves ocorrem caracter a caracter
- Árvore digital: estrutura de dados que permite realizar a busca em k passos, onde k é o tamanho da chave procurada

Árvore digital

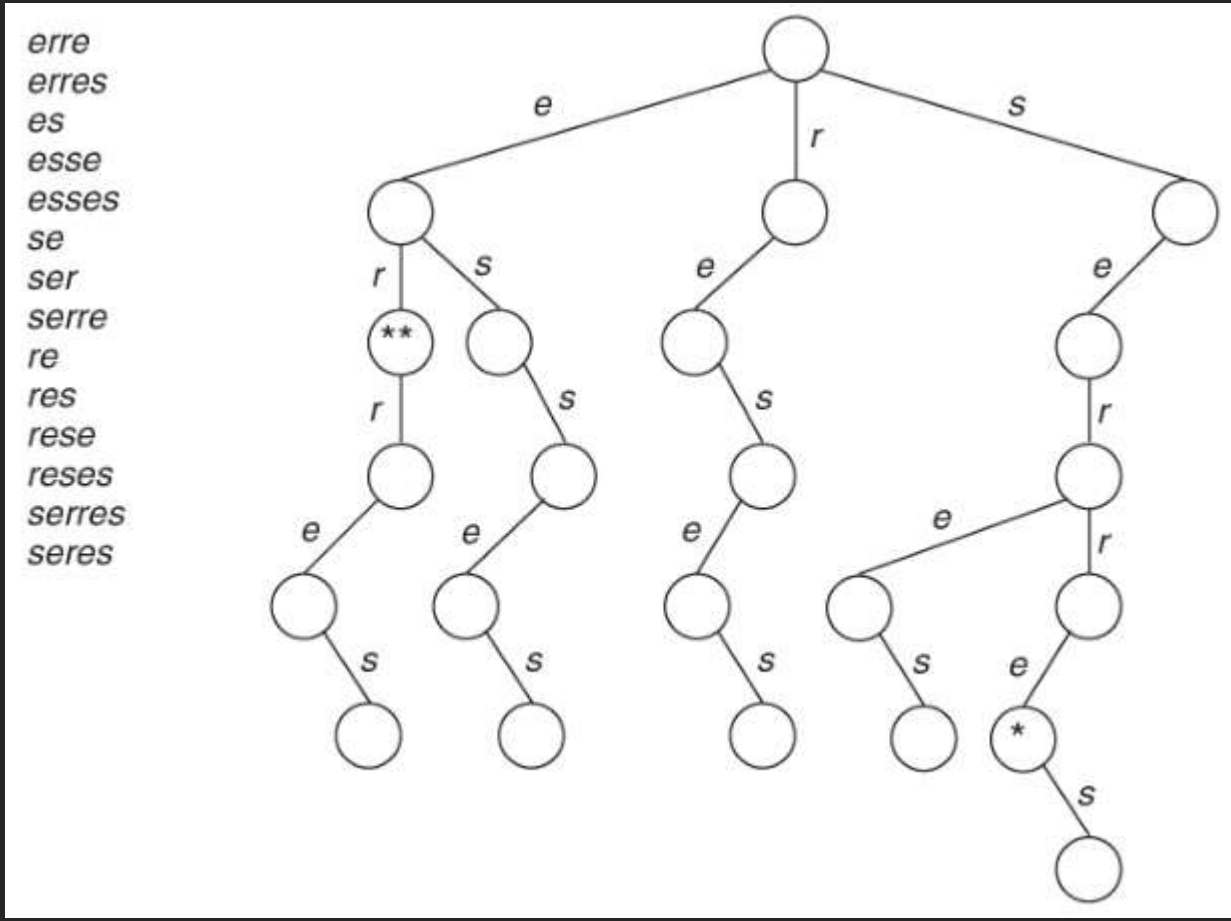
- Seja $S = \{s_1, ..., s_n\}$ um conjunto de n chaves onde cada s_i é formada por uma sequência de elementos d_j (dígitos)
  - Considere que em S há m dígitos distintos que formam o alfabeto de S
  - Dígitos de S admitem ordenação $d_1 < ... < d_m$
  - Os primeiros p dígitos de uma chave formam o prefixo de tamanho p da chave

Árvore digital

- Uma árvore digital para S é uma árvore m-ária T, onde:
  1. Se um nó v é o j-ésimo filho de seu pai, então v corresponde ao dígito d_j do alfabeto S, $1 \leq j \leq m$
  2. Para cada nó v, a sequência de dígitos definida pelo caminho desde a raiz de T até v corresponde a um prefixo de alguma chave de S
- A raiz de uma árvore digital sempre existe e não corresponde a qualquer dígito do alfabeto
- Nós terminais indicam o último dígito de uma chave válida

Árvore digital

- Ex.: árvore digital para o alfabeto {e, r, s}
  - m = 3
  - Dígitos ficam nas arestas
  - Nó terminal *: chave serre



Árvore digital

- Implementação do nó terminal: ponteiro
  - Aponta para demais informações relacionadas à chave
  - Ponteiro nulo: não é terminal
- Algoritmo de busca de uma chave de tamanho k
  - Realiza caminho da raiz até um nó de profundidade, no máximo, k
  - Complexidade: O(k log m)
    - Se valor de m é pequeno, dígito é usado como índice para determinar posição na ordenação do alfabeto: O(k)

Árvore digital

- Algoritmo de busca digital:
  - Se chave foi encontrada, então a = 1: senão, a = 0
  - Chamada externa: pt = ptraiz, l = 0, a = 0

```
algoritmo 9.1: Busca digital
procedimento buscadig(x, pt, ℓ, a)
    se ℓ < k então
        seja j a posição de d(ℓ + 1) na ordenação do alfabeto
        se pt ↑ .pont[j] ≠ λ então
            pt := pt ↑ .pont[j];   ℓ := ℓ + 1
            buscadig(x, pt, ℓ, a)
    senão se pt ↑ .info = terminal então a := 1
```

Árvore digital

- Algoritmo de inserção da chave x de tamanho k:
  - Busca x
  - Cria nós faltantes: a partir de onde a busca parou
  - Marca último nó como terminal
- Complexidade da inserção: $O(k_1 \log m + k_2 m)$
  - k_1: tamanho do prefixo x já existente na árvore
  - k_2: número de nós a serem incluídos
  - $k = k_1 + k_2$

Árvore digital

- Algoritmo de inserção:

algoritmo 9.2: Inclusão em árvore digital

$pt := ptraiz; \quad \ell := a := 0$

$buscadig(x, pt, \ell, a)$

se $a = 0$ **então**

  para $h = \ell + 1, \ldots, k$ **faça**

    seja j a posição de $d(h)$ na ordenação do alfabeto

    $ocupar(ptz)$

    para $i = 1, \ldots, m$ **faça** $ptz \uparrow .pont[i] := \lambda$

    $pt \uparrow .pont[j] := ptz; \quad ptz \uparrow .info := \textit{não terminal}$

    $pt := ptz$

  $pt \uparrow .info := \textit{terminal}$

senão "inclusão inválida"

Árvore digital

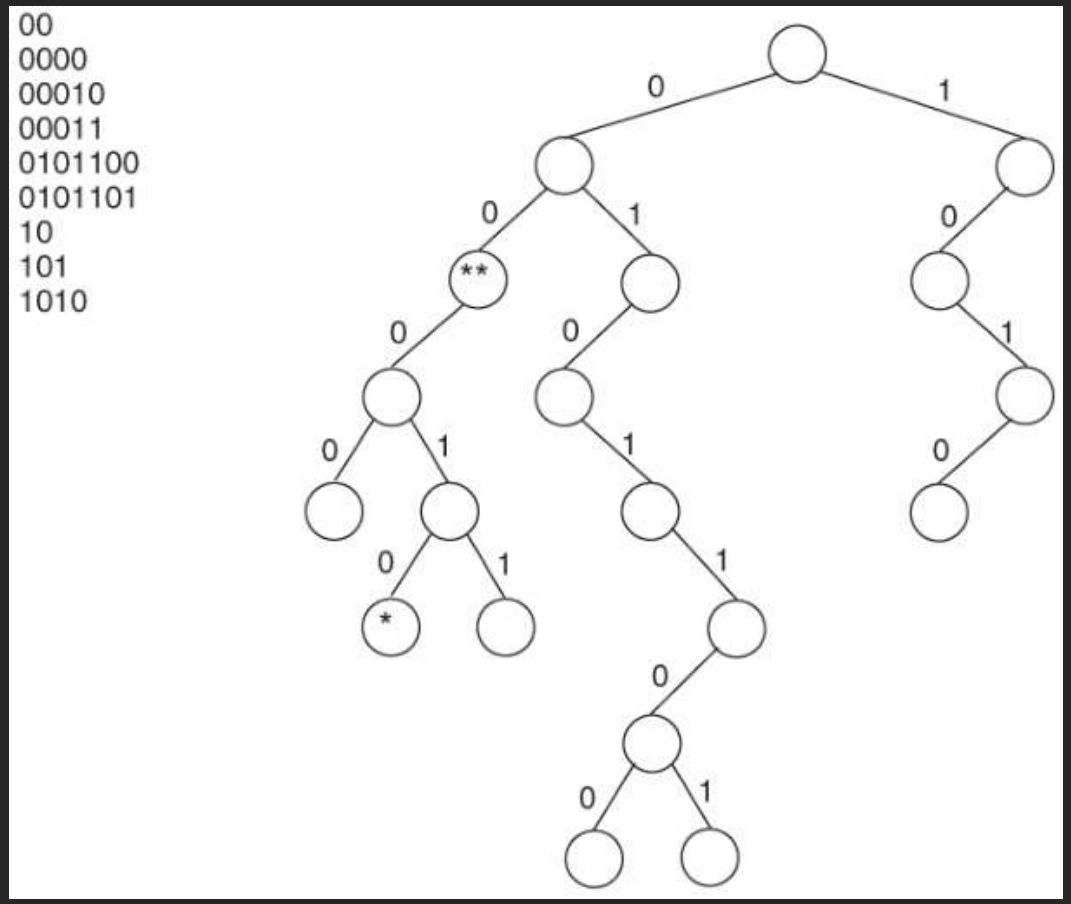
- Vantagens da busca digital:
  - Complexidade da busca não depende da quantidade de chaves armazenada: apenas do tamanho da chave buscada e do alfabeto
  - Chaves podem ter tamanhos arbitrários
- Desvantagem: excesso de memória utilizado
  - Necessita de O(n m t) para armazenar n chaves de tamanho t com prefixos distintos
  - Melhoria: eliminar ponteiros λ usando lista circular
    - Mas isso aumenta tempo de busca

Árvore digital binária

- Uma árvore digital binária é uma árvore digital com m = 2 e alfabeto {0, 1}
  - Filho esquerdo: dígito 0
  - Filho direito: dígito 1
  - Cada chave é uma sequência binária
  - É, possivelmente, o tipo de árvore digital mais usado:
    - Chaves possuem representação naturalmente binária
    - Quantidade de ponteiros λ é menor, em relação a árvores digitais com m > 2

Árvore digital binária

- Ex.: árvore digital binária



Árvore digital binária

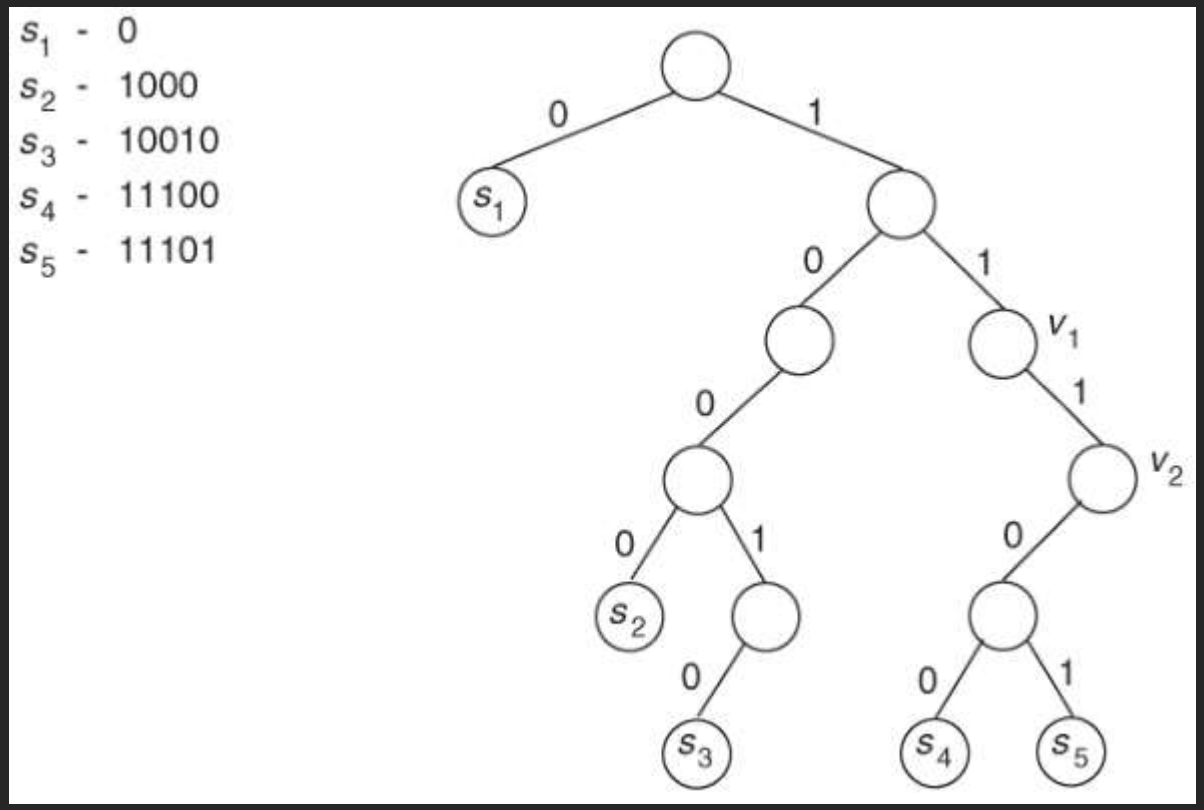
- Na árvore digital binária:
  - Ocorre formação de ziguezagues: indesejável
    - No exemplo anterior, para encontrar as chaves 0101100 ou 0101101 há um ziguezague do terceiro ao sexto nó
  - Uma chave pode ser prefixo de outras
    - No exemplo, a chave 00 é prefixo de 00010: o caminho da raiz até o nó ** é parte do caminho da raiz até o nó *

Árvore digital binária

- Uma árvore binária de prefixo é uma árvore digital binária onde nenhuma chave (código) é prefixo de outra
  - Propriedades:
    - Cada chave corresponde a uma folha
    - Codificação binária de uma chave: caminho da raiz até folha correspondente
    - Ainda pode ter ziguezague
  - Exemplo de aplicação: codificação de dados para compactação
    - Associa a cada dado um código apropriado

Árvore digital binária

- Ex.: árvore binária de prefixo



Árvore Patrícia

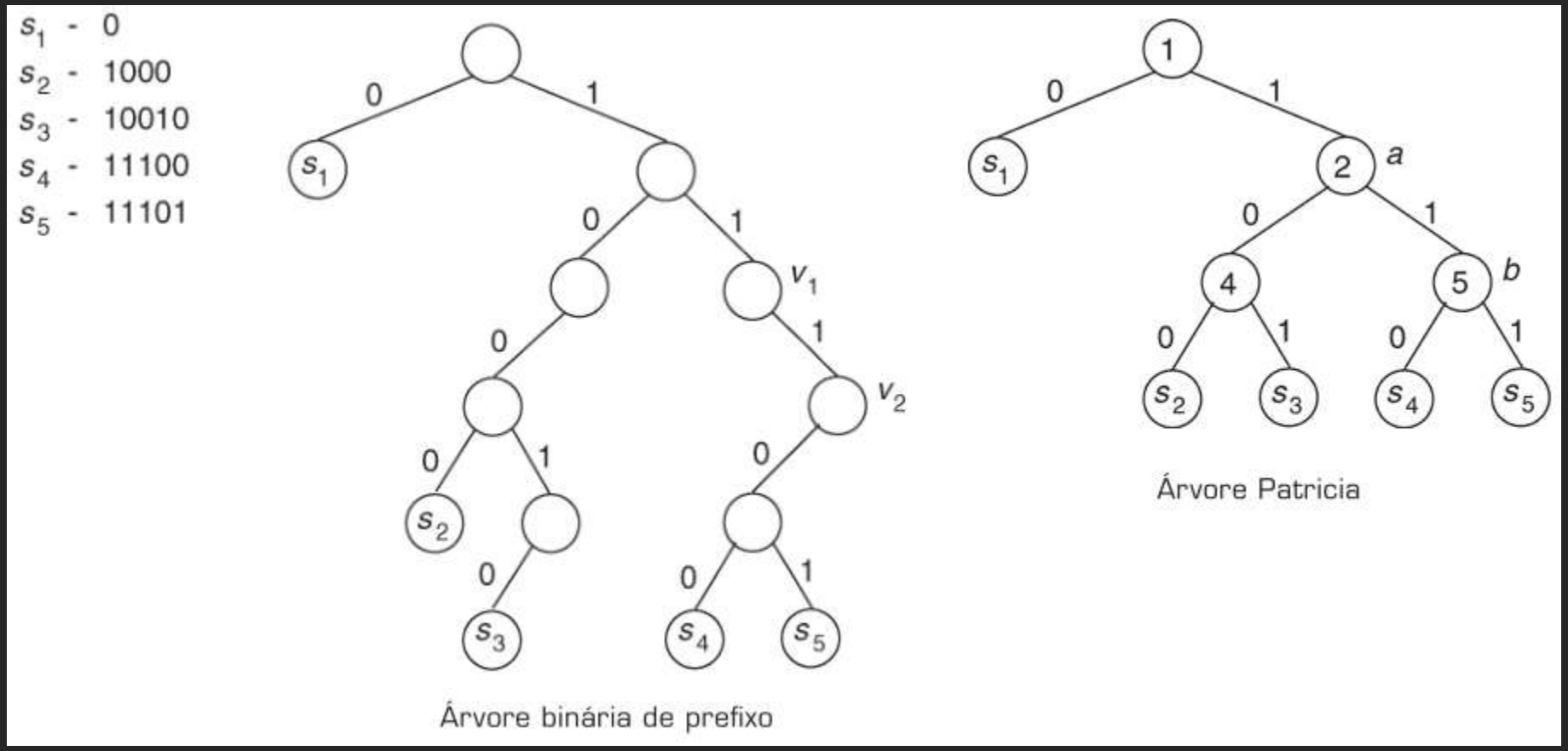
- Árvore Patrícia: árvore binária de prefixo com eliminação de ziguezagues
  - Practical Algorithm to Retrieve Information Coded in Alphanumeric
  - Idéia: juntar nós de ziguezague em um só
    - Nós são rotulados com essa informação
- Busca por $x = s_4 = 11100$ no exemplo anterior:
  - Após comparar os dois primeiros dígitos 11 não há alternativa além do ziguezague
  - Pode-se eliminar necessidade de testar o terceiro e o quarto dígito: “devem” ser 10 (se o código buscado é válido)

Árvore Patrícia

- Uma árvore binária de prefixo H pode ser convertida em uma árvore Patrícia T da seguinte forma:
  - Para cada ziguezague $v_1,\ldots,v_k$ de H:
    - Se v_k é folha:
      - Corresponde a uma chave s_i
      - Nós $v_1,\ldots,v_k$ são compactados em v_1 e $r(v_1) = s_i$
    - Se v_k não é folha, então v_k possui filho w:
      - Nós $v_1,\ldots,v_k$,w são compactados em v_1 e $r(v_1) = \text{nível}_H(v_1) + k$
  - Demais nós v em T que estão sem rótulo:
    - $r(v) = \text{nível}_H(v)$

Árvore Patrícia

- Ex: árvore Patrícia convertida de árvore binária de prefixo



Árvore Patrícia

- Uso dos rótulos na busca pela chave x:
  - Nó v não folha: r(v) indica o próximo dígito de x que deve ser comparado
  - Nós v folha: r(v) armazena o valor de uma chave
- Durante a busca da chave x, em cada nó v:
  - Se v é folha:
    - Se x = r(v): chave encontrada
    - Caso contrário, não encontrada
  - Se v é não folha, é testado o r(v)-ésimo dígito de x
    - Filho esquerdo (0) ou filho direito (1) é seguido
    - Se não existir o dígito r(v), isto é, k < r(v): chave não encontrada

Árvore Patrícia

- Algoritmo de busca em uma árvore Patrícia:
  - Chamada externa: buscapat(x, ptraiz, a)
  - Considera apenas chaves válidas: busca por 11011?

```
algoritmo 9.3: Busca em árvore Patrícia
procedimento buscapat(x, pt, a)
    se pt ↑ .esq = λ então a := 1
    senão se k < pt ↑ .r então a := 2
            senão se d[pt ↑ .r] = 0 então
                        pt := pt ↑ .esq
                        buscapat(x, pt, a)
                    senão pt := pt ↑ .dir
                        buscapat(x, pt, a)
```

Árvore Patrícia

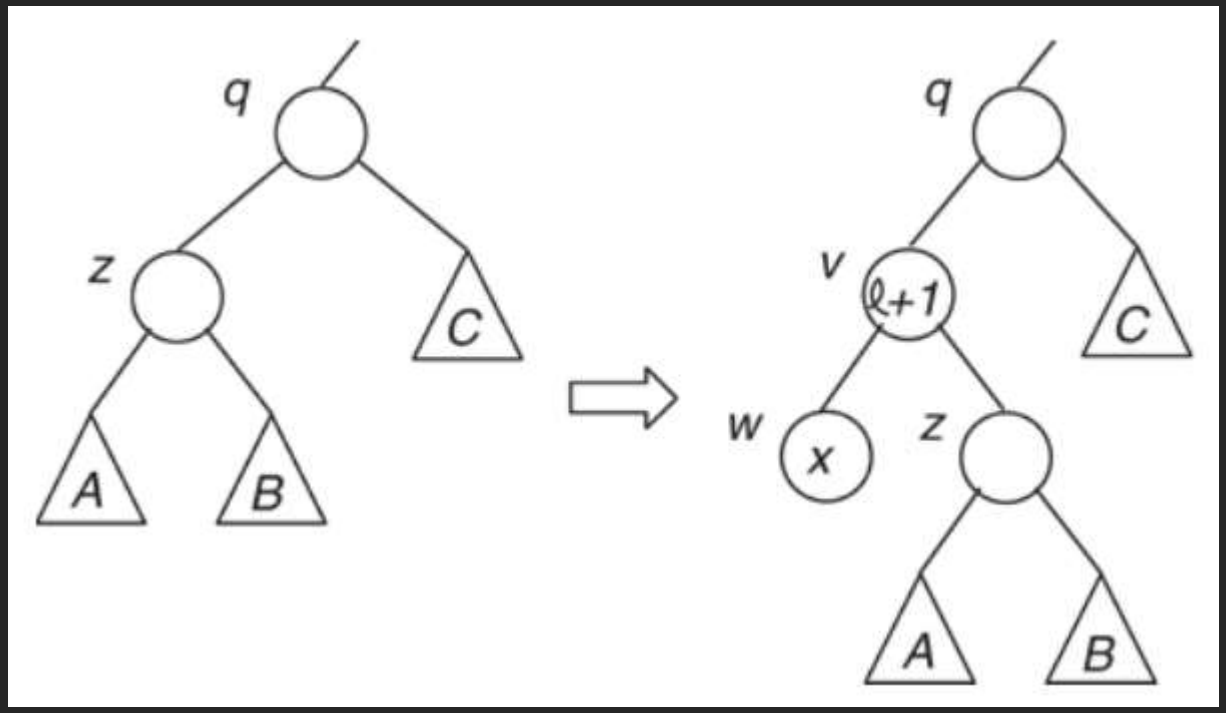
- Complexidade de tempo da busca:
  - Igual à altura da árvore
    - É reduzida pela eliminação de ziguezagues
  - Limitada pelo tamanho da maior chave
  - Se é feita comparação de x com chave na folha: proporcional ao tamanho da chave x
    - Se não é feita comparação: proporcional à altura da árvore
- Eliminação de ziguezagues pode não resultar em melhor complexidade de tempo mas resulta em menor uso de memória

Árvore Patrícia

- Inserção de x de k dígitos na árvore Patrícia T:
  - Busca x: termina em algum nó y de T
  - Seja s_i a chave correspondente a alguma folha y' descendente de y
    - Seja c a quantidade de dígitos de s_i
    - Seja l o tamanho do maior prefixo comum entre x e s_i
    - Se l = c ou l = k, a inclusão é inválida: uma é prefixo da outra
  - A inclusão ocorre no nó z:
    - Se y' é o único nó de T, z = y'
    - Senão, seja z' o pai de y' e A o caminho entre a raiz de T e z'
      - Se r(z') ≤ l+1, então z = y'
      - Senão, se r(z') > l+1, z = nó de A mais próximo à raiz onde r(z) > l+1
    - São criados dois nós, v e w, com rótulos r(v) = l+1 e r(w) = x
    - Os filhos de v são w (esquerdo, se d_{l+1} = 0, direito se = 1) e z. Se z era a raiz de T, a nova raiz passa a ser v

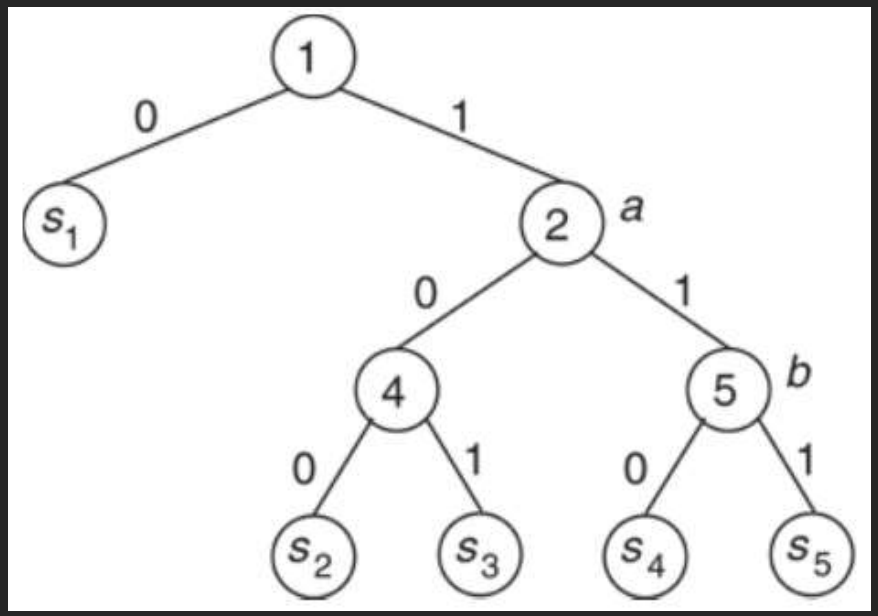
Árvore Patrícia

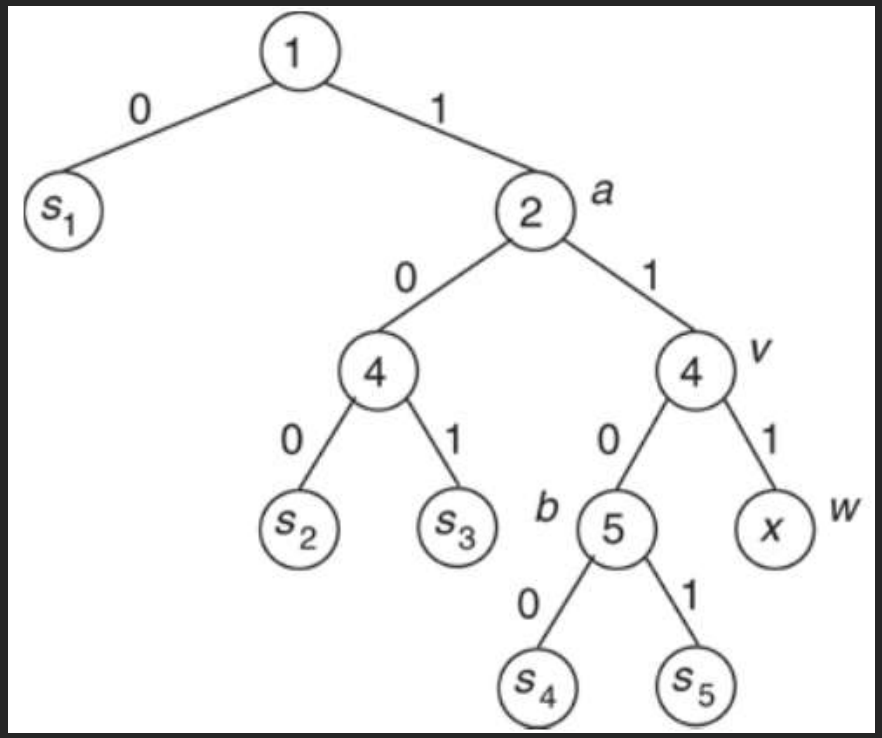
- Passo principal da inserção em árvore Patrícia: inserção em z



Árvore Patrícia

- Ex.: inclusão da chave 111101 na árvore Patrícia:





Exercícios

- 9.1 a 9.5